

ステップ4: タスク削除時に localStorageからも削除する

前のステップでは、タスクの追加、表示、完了状態の変更とそれらのlocalStorageへの保存・復元ができるようになりました。しかし、現状ではタスクを「削除」ボタンでリストから消しても、その変更はlocalStorageには反映されません。ページをリロードすると、削除したはずのタスクが復活してしまいます。

このステップでは、タスクを削除する際に、localStorageからも対応するタスク情報を削除するように `deleteTask` 関数を修正します。

学習目標

- 特定のタスクを削除した後、その変更をlocalStorageに正しく反映させる方法を理解する。
- `deleteTask` 関数内で `saveTasks` 関数を呼び出すことの重要性を再確認する。

1. `deleteTask` 関数の確認と修正

現在の `deleteTask` 関数は以下のようになっています（ステップ3完了時点）。

```
// script.js（ステップ3完了時点の抜粋）

// ...（他のコード）...

function deleteTask(index) {
  tasks.splice(index, 1); // 配列からタスクを削除
  // saveTasks(); // ← ★現状ここがコメントアウトされているか、または存在しない場合がある
  renderTasks();
}

// ...（他のコード）...
```

この関数は、`tasks` 配列から指定されたインデックスのタスクを `splice` メソッドで削除し、その後 `renderTasks` を呼び出して画面表示を更新しています。

localStorageに変更を反映させるためには、配列からタスクを削除した **後** に、現在の

`tasks` 配列の状態をlocalStorageに保存する必要があります。これは、すでにおなじみの

`saveTasks()` 関数を呼び出すことで実現できます。

修正後の

deleteTask

関数:

```
// script.js

// ... (既存のコード) ...

/**
 * タスクを削除する関数
 * @param {number} index - 削除するタスクのインデックス
 */
function deleteTask(index) {
  tasks.splice(index, 1); // 配列からタスクを削除
  saveTasks(); // ★変更点: 変更後のタスク配列をlocalStorageに保存
  renderTasks(); // 表示を更新
}
```

1.1. データフローと処理の図解（タスク削除時）

タスクが削除されてlocalStorageの内容が更新されるまでの流れは以下のようになります。

```
``mermaid
graph TD
  A[ユーザーが「削除」ボタンをクリック] --> B[イベントリスナー発火 (renderTasks内で設定)];
  B --> C[deleteTask関数実行 (該当タスクのindexと共に)];
  C --> D[tasks配列から該当タスクをspliceで削除];
  D --> E[saveTasks関数実行];
  E --> F[更新されたtasks配列をJSON文字列に変換];
  F --> G[localStorageに保存 (キー: todoTasks) - 実質的な上書き更新];
  G --> H[renderTasks関数実行];
  H --> I[画面のリスト全体を再描画 (削除されたタスクは表示されない)];
```

2. 動作確認

1. いくつかのタスクを追加します。
2. いずれかのタスクの「削除」ボタンをクリックします。タスクがリストから消えることを確認します。
3. ブラウザをリロード（再読み込み）します。
4. **期待される結果:** 手順2で削除したタスクは、リロード後も表示されない（localStorageから正しく削除されている）。

もし、リロード後に削除したはずのタスクが再表示される場合は、`deleteTask` 関数内で `saveTasks()` が正しく呼び出されているか、もう一度確認してください。

また、開発者ツールでlocalStorageの内容を確認するのも有効です。

- タスク削除前: localStorageに複数のタスクが保存されている。
- 特定のタスクを削除後: localStorageから該当タスクが消えている。

3. まとめ

このステップでは、`deleteTask` 関数に `saveTasks()` の呼び出しを追加するだけで、タスク削除の変更をlocalStorageに正しく反映させることができました。

これにより、タスクのCRUD（Create: 作成, Read: 読み取り, Update: 更新, Delete: 削除）操作すべてにおいて、変更がlocalStorageに永続化されるようになりました。

- **Create** : `addTask` 関数で新しいタスクを追加し、`saveTasks` で保存。
- **Read** : `loadTasks` 関数でlocalStorageから読み込み、`renderTasks` で表示。
- **Update** : `toggleTaskCompletion` 関数で完了状態を更新し、`saveTasks` で保存。
- **Delete** : `deleteTask` 関数でタスクを削除し、`saveTasks` で変更を保存。

これで、ローカル永続化を行う基本的な「やることリスト」サイトの機能が一通り完成しました。

4. Q&A

Q1: `deleteTask` で `saveTasks()` を呼び出すのを忘れるとどうなりますか？

A1: 画面上ではタスクが削除されたように見えます（`renderTasks()` が呼び出されるため）。しかし、`tasks` 配列の変更が `localStorage` に保存されないため、ページをリロードすると削除したはずのタスクが元に戻ってしまいます。

Q2: `tasks.splice(index, 1)` は何をしていますのですか？

A2: `splice` はJavaScriptの配列メソッドで、配列の既存の要素を取り除いたり、置き換えたり、新しい要素を追加したりすることができます。 - `tasks.splice(index, 1)` の場合: -
`index` : 操作を開始する位置（インデックス）。 - `1` : 取り除く要素の数。つまり、`tasks` 配列の `index` 番目の位置から1つの要素を取り除く、という意味になります。これにより、指定されたタスクが配列から削除されます。

Q3: もしタスクが非常に多い場合、毎回 `saveTasks()` で全データを保存し直すのは非効率ではないですか？

A3: はい、その通りです。タスクの数が非常に多くなり、パフォーマンスが問題になるような状況では、より効率的な更新方法を検討する必要があります。例えば、変更があったタスクだけを特定して `localStorage` 内の対応する部分のみを更新する、あるいは `IndexedDB` のようなより高機能なブラウーストレージを利用するなどの方法が考えられます。しかし、この演習の範囲では、`localStorage` の基本的な使い方とデータ永続化の概念を理解することを目的としているため、シンプルに全データを再保存する方法を採用しています。一般的なToDoリスト程度のデータ量であれば、この方法でも通常は問題ありません。

5. コラム : localStorage.removeItem() について

今回の演習では、タスクリスト全体を `tasks` 配列として管理し、変更があるたびに `tasks` 配列全体を `localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks))` で上書き保存しています。そのため、特定のタスクを削除する際も、配列から要素を削除した後に配列全体を再保存するアプローチを取りました。

`localStorage` には、特定のキーに保存されたアイテムを直接削除するための `localStorage.removeItem(key)` というメソッドも存在します。

もし、各タスクを個別のキーで `localStorage` に保存するような設計（例： `todo-task-1` , `todo-task-2` のように）にしていた場合は、タスク削除時に `localStorage.removeItem('todo-task-X')` のようにして個別に削除する必要が出てくるでしょう。

しかし、今回の演習のように単一のキーで配列全体を管理する方法は、多くのケースでシンプルかつ効果的です。

次のステップ（発展）では、タスクの内容を編集する機能について考えてみましょう。